

SCHEMADOC AI

AI-Powered Database Schema Analyzer

*Upload a database schema. Generate enterprise documentation.
Share a printable PDF with stakeholders.*

Client Requirements & Solution Brief

Prepared for: Client Review & Discussion
Version: 1.0 | **Date:** June 2026
Classification: Confidential

1. Executive summary

SchemaDoc AI is a web application that ingests a database schema file and automatically produces comprehensive software documentation, business analysis artifacts, and system design outputs. The application combines the perspectives of a Business Analyst, Solution Architect, Product Owner, QA Lead, and Technical Architect.

Per client requirements, the solution uses a **Python** API backend, a **React** web UI, **no application database**, and stores all data as **text and JSON files** on the filesystem. The primary deliverable for client meetings is a **printable PDF report** generated from the analysis.

2. Agreed client constraints

Requirement	Our approach
No application database	Zero DB — no PostgreSQL, SQL Server, or SQLite for app data
File-based storage	Projects, schemas, and deliverables saved as JSON/Markdown/text files
Python backend	FastAPI for parsing, AI orchestration, and PDF generation
React UI	Modern responsive interface for upload, review, and export
Client PDF	One-click printable report for stakeholder discussions

3. How it works

Upload. User uploads SQL DDL, schema JSON, or supported ERD export. System validates and parses tables, columns, keys, constraints, indexes, views, and routines.

Analyze. AI classifies structure, detects business domain, clusters modules, and generates deliverables grounded in schema evidence.

Review. User browses results in the React dashboard — features, FRD, user stories, screens, APIs, gaps, and more.

Share. User downloads a **Client Discussion PDF** formatted for A4 printing — ready for executive and technical review meetings.

4. Generated deliverables

All outputs are derived from the uploaded schema. No placeholder text. Each item cites related tables and schema evidence.

#	Artifact	Contents
1	Feature Inventory	Features, tables, complexity, business value
2	FRD	Functional modules with inputs, rules, outputs, validation
3	User Stories	Agile format with acceptance criteria and story points
4	UI Screen List	Screens, controls, grid columns, actions
5	Menu Structure	Hierarchical navigation tree
6	Role Permission Matrix	CRUD, Export, Approve by role and screen
7	Business Workflows	Mermaid process flowcharts
8	ER Diagram	Entity relationships with cardinality
9	API Requirements	REST endpoints + OpenAPI specification
10	Test Cases	Functional, integration, security, performance
11	PRD	Vision, personas, goals, risks, success metrics
12	Gap Analysis	Missing audit, security, indexes, archival patterns

Additional AI outputs:

- Domain summary with confidence score
- Project complexity score and effort estimates
- Architecture recommendations (monolith vs microservices, auth, caching)
- Security review (PII exposure, audit gaps, authorization model)

5. Client PDF report (primary shareable artifact)

The PDF is designed for meetings where stakeholders may not have access to the web application. It is professionally formatted for A4 printing with cover page, table of contents, and page numbers.

PDF contents:

- Executive summary — domain, complexity, top findings
- Schema overview — table count, relationships, ER diagram
- Feature inventory and gap analysis (severity-rated)
- Sample user stories, UI screens, and API overview
- Architecture and security recommendations
- Appendix — full table list and methodology notes

6. Technology stack

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS
Backend	Python 3.11+, FastAPI, Pydantic
AI	OpenAI API with structured JSON output
Storage	Filesystem — JSON, Markdown, text files only
PDF export	Python-generated printable report
Deployment	Docker Compose with shared data volume (optional)

Explicitly excluded in v1: application database, complex multi-tenant auth, live database connectors.

7. Dashboard metrics

- Tables and relationships count
- Features, screens, APIs, and user stories identified
- Gaps identified by severity (Critical / High / Medium / Low)
- Business domain detected with confidence percentage
- Overall complexity score

8. Quality standards

- **Schema-grounded** — every output justified by table/column/FK evidence
- **No placeholders** — enterprise-grade BA, Architect, PO, and QA quality
- **Consistent** — same module and feature names across all artifacts
- **Traceable** — deliverables stored as readable JSON/text files on disk

9. Phased delivery

Phase 1 — MVP

- Schema upload and parsing (PostgreSQL + SQL Server minimum)
- AI analysis and core deliverables (features, FRD, stories, screens, gaps, ER)
- React UI with dashboard and deliverable pages
- Client PDF, Markdown, and HTML export

Phase 2

- Full API spec, workflows, interactive ER diagram, DOCX/XLSX export

Phase 3

- Analysis versioning, domain templates, optional authentication

10. Phase 1 success criteria

1. Client provides or approves a sample schema (30–50 tables)
2. Upload and parse complete with clear error reporting on failure
3. AI analysis produces all Phase 1 deliverables as files on disk
4. React UI displays dashboard metrics and deliverable content
5. **Client PDF downloads and prints cleanly for stakeholder review**
6. Confirmed: no database used anywhere in the application

Upload once. Document everything. Share a PDF your stakeholders can discuss tomorrow.

11. Recommended next steps

1. Confirm sample schema (or use provided healthcare/laboratory sample)
2. Approve Phase 1 scope and Client PDF outline
3. Provision OpenAI API access and deployment environment (Docker or VM)
4. Schedule MVP demo and first Client PDF review meeting